Verified Search with Neural Ranking and Executable Claims

Rules decide what is allowed; neural models rank what remains

Anonymous authors

Abstract. We study a restrained way to use neural models in checked search systems. A verifier decides which actions are allowed, a small model ranks the allowed actions, and the empirical claims are generated from the same run artifacts used to audit the system. The contribution is not that tiny networks solve hard tasks. It is that checked failures and checked successes become reusable training and proposal structure without giving the model authority over correctness. The clean anchor is arithmetic sequence filtering: in the audited next-prime instance (`prime_gap` in traces), the generated claim row reports the verified learned filter raises top-1 ranking to 68.87% from 45.12% for the hand-coded wheel and 9.13% raw, with 0 verifier-counted true-answer deletions. The same family also admits recurrence filters such as Fibonacci/Lucas gates via period residues, but those remain smoke-level boundary evidence here. The main feedback results are rejected-candidate reason learning and certified hypothesis-library promotion. Boundary rows for ARC, Signal, tiny-net capacity, and haiku test where this protocol holds and where it weakens. The current artifact also exposes a dataset-first engine surface that ports several domains into one candidate-row contract, but that surface is infrastructure evidence rather than a new domain-result claim. The scope of this evidence explicitly excludes broad ARC competence, CDCL-equivalent learned clauses, comparative live neural-control improvement, semantic language understanding, and general end-to-end task solving.

1. Introduction

Many tasks have a simple asymmetry: it is hard to know the best action in advance, but easy to check whether a proposed action is allowed. In those settings, the learned model should not own the final answer. A rule, verifier, certificate, or replay system should decide what is valid. The model should only decide which valid candidate to try first.

The same discipline should govern the claims. Reported numbers should come from trace rows, environment identity, checkpoint hashes, result summaries, and generated limitation candidates rather than from hand-maintained prose.

The thesis is:

"Let exact or replayable checks decide what is allowed. Let small learned models order the allowed choices. Let repository artifacts decide which empirical claims are admissible."

The operational summary is simple: the model proposes, the checker decides, and the trace remembers enough to train the next proposal pass and constrain the next empirical claim.

Two empirical loops carry the result. First, rejected verifier rows are logged with reason codes and reused as supervision for a later safety-first residual ranker. Second, repeatedly certified hypotheses are promoted into a library and reused as proposal priors under held-out budgets. The arithmetic sequence-filter result supplies the clean anchor; the fixed-budget proposer failure supplies the falsifying negative row that motivates the feedback loop.

The remaining domains are boundary evidence. ARC tests grammar coverage and failure taxonomy, not benchmark competence. Signal tests replay and guarded live execution pressure, not comparative live-control performance. Tiny-net and adapter rows bound architecture claims. Haiku is a language-adjacent corpus check, not semantic understanding. The dataset-first runner is a shared infrastructure surface: it can show that a domain compiles to complete candidate rows, verifier labels, traces, checkpoints, and engine evidence, but it does not by itself promote the domain's scientific result. New domains stay outside the claim set until promoted through the evidence matrix.

The environment names are deliberately small. The audited `prime_gap` trace is the next-prime instance of a broader modular-filter family; smoke rows already include Fibonacci/Lucas recurrence filters via period residues. `grid_path` tests topological path constraints; delayed-claim tests hidden outcome and safety-margin gates; ARC tests visual-spatial abstraction provenance; Signal tests bounded WASD control under replay and live telemetry; haiku tests corpus-derived proxy grounding.

At this artifact snapshot, the generated claim compiler reports 14 supported claims, 0 contradicted claims, 4 underpowered claims, 0 missing claims, and 0 stale claims. Contradicted and underpowered rows remain visible because falsifiability is part of the protocol. The fixed-budget proposer row is a supported negative baseline (the static fixed-budget proposer is a supported negative baseline: candidate static saves 72.49 vs best deterministic 73.22, and conditional static saves 2.15 vs best deterministic 55.57.), and the tiny-net plateau row is underpowered (tiny-net plateau audit covers 6 capacities, 3 restarts, and 8 checkpoints through 128 epochs across 12 domain/variant verdicts; 20/72 required rows meet the plateau threshold and 0/12 verdicts are audit-ready.).

2. Positioning

The formal-methods ancestry is counterexample-guided search. Sketching combines inductive synthesis with validation-generated counterexamples [1], and SyGuS names the grammar-plus-specification setting [2]. CEGAR and CDCL add the adjacent lesson that failed traces can refine later search rather than disappear as waste [3], [4], [5]. This protocol inherits that shape but uses failures as supervised ranking data, not as sound learned clauses.

Modern verifier-bounded learning asks models to solve or help verify reasoning traces [6], [7], [8], [9], [10]. AlphaGeometry-style systems combine learned proposal with formal checking [11], [12], and theorem-proving work similarly uses neural proposal around formal environments [13]. The artifact here is smaller and narrower: the learned model never owns correctness.

The accountability lineage is equally important. Model Cards, Datasheets, reproducibility checklists, and reinforcement-learning reproducibility critiques made documentation and artifact discipline central research objects [14], [15], [16], [17]. Here, that idea becomes a build rule: numbers, limitations, and support status are generated from repository artifacts before claim compilation.

3. Protocol

The protocol has three jobs. First, list the candidates. Second, use a checker or replay to decide what each candidate is allowed to mean. Third, let a small model rank only the remaining choices. Every promoted row records those jobs in the same contract:

```
Task T = {
  state distribution S_T,
  candidate enumerator A_T(s),
  observable features x_T(s, a),
  candidate ranker f_theta(x_T),
  checking or replay operator O_T(s, a),
```

```
  checker authority level,
  grammar G_T,
  certificate or replay checker C_T,
  trace / replay / claim protocol
}
```

The protocol's invariant is simple:

`The model proposes. The checker decides. The trace remembers.`

Stage	**Concrete arithmetic-filter object**	**Artifact that survives**
Candidate	The proposer emits a modular gate candidate such as "reject candidate `n` when `n > 3` and `n % 3 == 0`" for next-prime search states.	Grammar id, candidate id, split, seed, and support window.
Gate check	The segmented-sieve verifier checks whether the rule removes any true next-prime outcome under the declared support.	A false-rejection certificate; accepted only when false rejections are zero.
Residual rank	Certified composite candidates are removed, and the model ranks the remaining action ids.	Top-k metrics, stable action ids, ranking policy, and score margins.
Trace emit	The run writes comparison, rule-search, and certificate-trace rows.	`crlp.compare.v1`, `crlp.rule_search.summary.v1`, environment and model hashes.
Claim consume	The claim compiler derives the registered summary from the artifact.	`claim_C1_prime_gate_delta_summary` and generated support/limitation status.

Table 1: Concrete arithmetic-filter trace. The headline number is downstream of candidate generation, verifier certification, model ranking, trace emission, and generated claim consumption.

Not every domain gives the system the same kind of checker. Exact verification is strongest. Replay is weaker but still auditable. Corpus tests are weaker again: they can show fit to a dataset, not truth. Claim scope changes with that authority level.

Authority	**Operator**	**Examples**	**Allowed claim**
exact check	proof, lookup, type check, deterministic verifier	`prime_gap`, `grid_path`, certified ARC transform rows	correctness and zero-false-rejection claims inside declared support
deterministic replay	simulator or replay deciding candidate consequences	Signal flight, delayed-claim simulator	replay-certified utility, regret, damage, death, or decision metrics
corpus proxy	train-split grammar, adjacency, corruption benchmark	haiku prototypes, public corpus coverage surfaces	corpus fit and near-miss discrimination, not semantic truth
human review	reviewable acceptance/rejection	editorial-agent pull requests	review-gated assistance, not autonomous empirical authority

Table 2: Checker authority levels. The same model ranking has different evidentiary force depending on which operator produced the outcome row.

The weaker operators do not inherit the authority of stronger ones. If `O_T` is learned, sampled, corpus-derived, or human-adjudicated, the checked object is the declared estimator or review process: its split, seed, rubric, artifact hash, and acceptance rule. The claim compiler may then certify conformance to that operator, not correctness in the underlying open domain. A human-reviewed claim therefore remains review-gated assistance; a corpus-proxy claim remains corpus fit; neither becomes an exact-verifier claim by passing through the same trace format.

The task-independent rules are short. Enumerate candidates before checking them. Say where features came from. Treat checker-derived quantities as labels, rewards, diagnostics, or gates unless a schema explicitly allows them as model inputs. Let rules enter the gate only through the task grammar and certificate or replay checker. JSONL traces carry stable candidate ids, ranking policy, score margins, environment identity, and semantic hashes so reordered candidates or float ties cannot silently change registered claim meaning.

4. Engine Substrate

The implementation has a lower-level substrate beneath the domain reports. A dataset-first action-scoring runner consumes grouped candidate rows:

```
split, group_id, state_hash, action_id, target, f0, ..., fN
```

Each group is one state with a complete candidate set. The target field is verifier-owned and may mark more than one valid candidate, so equivalent macro steps, tied shortest paths, corpus-valid continuations, or equal-utility actions need not be collapsed into an arbitrary single gold row. Domain adapters own state generation, feature encoding, candidate enumeration, and verifier labeling. The shared engine owns softmax ranking, trace emission, checkpointing, numeric summaries, and common engine-evidence rows.

The row contract also has a scaling boundary. It assumes the adapter can present a bounded candidate set for each state. Continuous-control, robotics, or multi-agent environments must therefore add a discretizer, sampler, motion-primitive library, or replayable proposal generator before they fit the runner. The guarantee is then over the enumerated candidate set and declared support, not over every action in the unconstrained environment. If the adapter cannot make that support explicit, the shared engine is instrumentation rather than a verified-search guarantee.

The generated engine claim is: the dataset-first engine evidence surface has 8 promoted-smoke rows across 8 domains; 8/8 rows declare dataset, rule-search, replay, and evidence capabilities, 8/8 pass candidate-group, verifier-target, and checkpoint-identity contracts, 4 rows expose multi-target verifier labels, and the weakest smoke test top-1 is 60.00%. These rows are infrastructure evidence, not promoted domain-result claims.

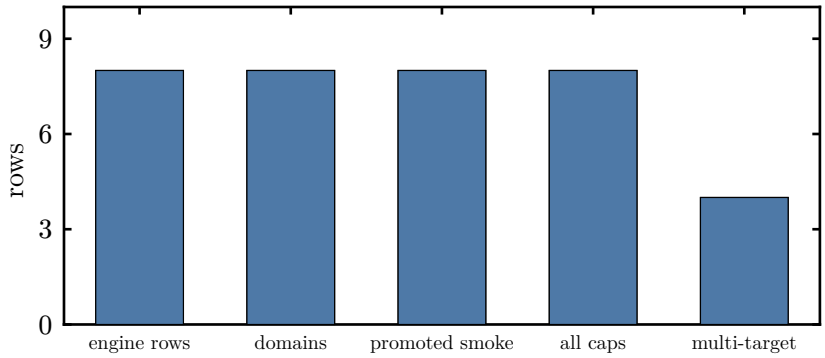


Figure 1: Dataset-first engine evidence surface. These rows prove that the registered smoke adapters can compile to one candidate-row protocol with replay/schema declarations; they do not replace richer domain-specific reports or promote broad task competence.

The generic trace-replay path is intentionally weaker than a domain-lossless replay adapter. When old traces do not store original state/action features, replay training uses trace-native features such as logged score, probability, action id, candidate count, and rank. That is enough to exercise stable action ids, multi-target labels, checkpointing, and numeric certificates. It is not evidence that grid, ARC, haiku, Signal, or program-synthesis semantics can be reconstructed from a score trace alone.

5. Evidence Matrix

The matrix is the scope boundary. Every row answers the same questions: what produced the candidates, what checked them, what did the model rank, what artifact proves the result, and what claim is allowed? A row may support a claim only if those pieces and its limitation are registered. New experiment families require revising this matrix first.

Family	Grounding	Learned Role	Claim / Boundary
arithmetic sequence filtering	**Grounding:** segmented-sieve replay and modular certificate checks zero-false-rejection divisibility certificates for the audited next-prime instance	rank residual gaps and budgeted certified filters	**Claim:** certified modular filters shrink the residual action space. **Limit:** static neural proposer is not superior to deterministic baselines at fixed budget
grid path	**Grounding:** BFS shortest-path successor and rule certificate checks legality, trap, clear-path, and distance-dominance certificates	rank legal residual moves after symbolic constraints	**Claim:** same gate contract works outside residue arithmetic. **Limit:** supporting transfer surface, not a main contribution
delayed claim	**Grounding:** deterministic hidden-outcome simulator and rule certificate checks finite-generator safety check plus false-rejection counters	rank utility only after safety dominates the objective	**Claim:** reason learning needs domain-aware safety constraints. **Limit:** reason-prior transfer alone can fail safety; claim remains domain-scoped
rejected candidate learning	**Grounding:** original rule/certificate checker that rejected or accepted each row false-rejection-first budget filter over reason priors and residual scores	learn residual ordering from rejection reasons and marginal verifier utility	**Claim:** verifier failures become training signal. **Limit:** not a generic reason-code ranker; delayed-claim safety remains scoped
hypothesis library feedback	**Grounding:** train-pair exactness, type checks, and held-out fixture split evaluation zero-false-rejection promotion before a binding enters the library	rank promoted slot priors and residual program candidates under tight budgets	**Claim:** certified learning can accumulate through reusable library priors. **Limit:** public ARC promotion remains scoped evidence, not a headline ARC score
multitask tiny net	**Grounding:** task-local labels and certified downstream gates per-task held-out metrics, OOD ARC variants, calibration bins	shared residual scorer with domain-specific diagnostics	**Claim:** representation/protocol limits dominate naive width scaling. **Limit:** architecture-size claims stay gated by plateau-ready diagnostics
arc agi evidence surface	**Grounding:** train-pair consistency, unique compiled prediction, and public corpus loader program/hypothesis certificate rows and scorer-invariant loader summaries	rank bounded candidates; scorer does not own correctness	**Claim:** ARC exposes priors, grammar coverage, and failure taxonomy. **Limit:** not a broad ARC competence or benchmark-score claim
arc verified self play	**Grounding:** known-correct synthetic output construction, ARC visual-pipeline verifier fields, and full/matched visual-only public ARC replay logs accepted rows are generated by primitive composition; public coverage requires ARC verifier replay against full and matched visual-only baseline logs	train inverse visual-program induction from exact self-play rows	**Claim:** ARC proposer training can be generated from verifier-owned primitives, but support requires fixed-budget public replay rather than synthetic in-distribution top-1. **Limit:** distribution mismatch, leakage audit, and public fixed-budget attribution remain open before self-play can be credited
signal flight	**Grounding:** offline counterfactual simulator replay, closed-loop rollout replay, live shadow frame comparison, and live active telemetry replay utility, regret, damage/death counters, certified control-target rollout rows, shadow disagreement logs, live safety gates, and protocol compatibility rows	rank bounded control candidates beside heuristic control	**Claim:** application pressure for replay-grounded residual ranking and guarded live neural-active execution. **Limit:** active neural versus heuristic performance is not yet claimed

Table 3: Frozen evidence matrix. Each row states the checker, learned role, supported claim, and boundary condition.

6. Anchor: Arithmetic Sequence Filtering

The cleanest test case involves arithmetic sequence filtering. The model suggests mathematical rules that filter out candidates known not to belong to a target sequence. While the family extends to sequences such as Fibonacci and Lucas numbers through Pisano-period residue gates, the strictly audited anchor here is a next-prime filter. A rule is only accepted if an exact mathematical checker confirms that it never deletes the true next-prime answer inside the declared support. The machine-readable trace id remains `prime_gap`.

In formal terms, candidate actions are possible next-prime gaps after a segmented-sieve window. Candidate filters are modular rejection predicates. A filter is accepted only if its certificate has zero false rejections; the scorer then ranks the remaining candidates. The `hand wheel` baseline is a deterministic hand-coded wheel-factorization filter over small modular divisibility rules.

Condition	Top-1	Top-10	False rejections
no gate	9.13%	78.01%	0
hand wheel	45.12%	99.77%	0
learned wheel	**68.87%**	**100%**	**0**

Table 4: Arithmetic sequence filtering on the canonical next-prime sample of 30000 rows: the verified learned filter raises top-1 ranking to 68.87% from 45.12% for the hand-coded wheel and 9.13% raw, with 0 verifier-counted true-answer deletions. Learned-vs-hand top-1 delta: +23.74 pp.

The seed audit checks that the headline does not depend on one lucky training run: across 5 seeds, the learned filter stays at 68.87% +/- 0.00pp, versus 45.12% +/- 0.00pp for the hand wheel and 9.13% +/- 0.00pp raw; the worst learned-vs-hand gain is +23.74pp, learned top-10 never falls below 100.00%, and false rejections never exceed 0; promotion requires 5 seeds. The zero spread is a seed-stability audit over this deterministic benchmark: seeds vary checkpoints and proposer runs, not the declared prime-range evaluation distribution.

This result is intentionally narrow. It shows that verified structure can shrink the search space sharply while preserving correctness under the declared support. It does not show that a static neural proposer beats deterministic symbolic ordering. The generated negative row says the opposite: the static fixed-budget proposer is a supported negative baseline: candidate static saves 72.49 vs best deterministic 73.22, and conditional static saves 2.15 vs best deterministic 55.57. That failure is useful because it prevents the claim set from becoming a generic "neural proposal wins" story.

The naive learned proposer fails. That is the point. Unsupported neural proposal is not trusted; verifier feedback is recycled until proposal economics improve. In this protocol, negative rows are not cleanup work after the fact. They are the training substrate that separates model ranking from validity.

7. Feedback Loops

The main contribution is what happens after the checker has said yes or no.

Rejected-candidate learning. When the verifier rejects a model suggestion, the system logs exactly why it failed. Those failures are then recycled as training data, teaching the model to rank future guesses better. In formal terms, failed rule proposals are logged with task, split, seed, reason family, false-rejection counts, and marginal verifier utility, then used as targets for a later ranker. The supported claim is scoped to arithmetic-sequence/modular sweeps: logging why the verifier rejected a guess repairs the next-prime proposer at budget 8: the unguided model chooses 7 unsafe rows and causes 172 validation false rejections, while the reason prior reaches 6 accepted rules/216.0 utility and the trained repair reaches 7 accepted rules/245.0 utility with zero false rejections; arithmetic-sequence/modular split sweeps pass, but the delayed-claim stress case has 0 mixed split rows.

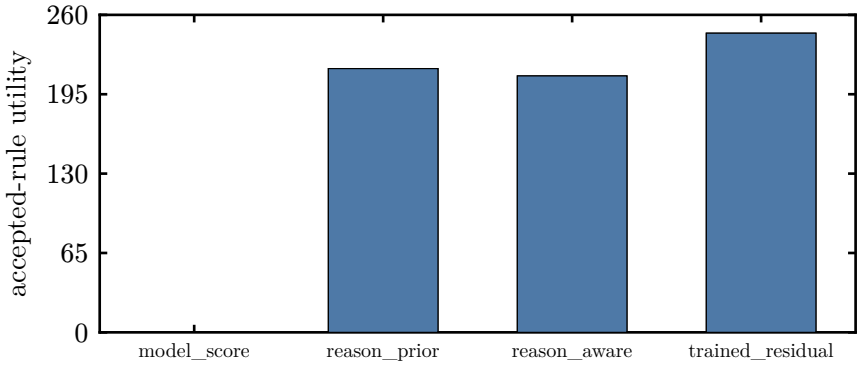


Figure 2: Rejected-candidate reason learning. Verifier failures become training signal without making the learned model authoritative.

Certified hypothesis-library feedback. If a generated rule is perfectly safe and verified multiple times, it is saved as a reusable building block. The model can then use those saved rules as proposal hints on future, unseen tasks. In formal terms, repeated zero-false-rejection hypotheses are promoted into a reusable library, converted into slot priors, and checked on held-out tasks under the same tight budget. The supported claim is: saving repeatedly safe rules helps future tasks: promoted move_to_anchor from 2 tasks/78 covered events with zero false rejections; slot priors raise first-choice certifications from 2 to 4 and budget-8 utility from 0.2119 to 0.5169. Held-out split feedback rows: 30.

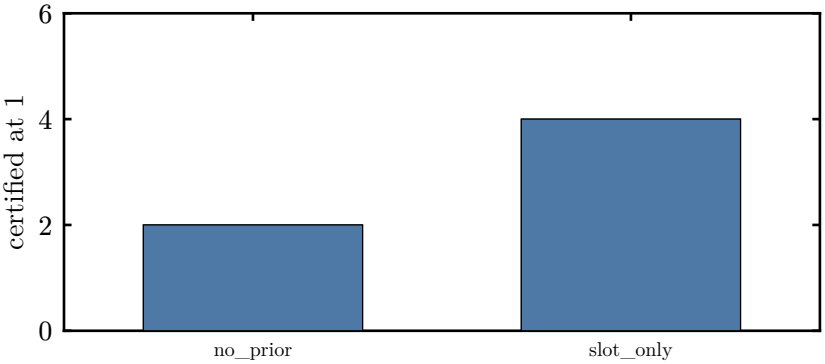


Figure 3: Certified hypothesis-library promotion. Certified hypotheses become proposal priors only after passing the promotion gate.

Together these rows are the protocol's feedback mechanism. Rejections and certifications are not after-the-fact diagnostics; they are reusable artifacts that change the next proposal pass. They are also artifact-bounded, not an online self-modifying policy. Promotion uses frozen artifact identity, split and seed labels, zero-false-rejection gates, held-out checks, and semantic trace hashes. Continuous monitoring can rebuild summaries and limitation candidates, but a changed checkpoint or trace snapshot creates a new artifact identity rather than silently updating an existing claim. This keeps feedback useful while making policy drift and rebuild cost visible in the evidence surface.

8. Boundary Evidence

Boundary rows keep the thesis honest without becoming separate stories.

ARC. ARC is a provenance and grammar-coverage stress surface. The reported result is the current measured public-corpus surface, not a headline ARC score: the ARC row measures coverage, not competence: public eval coverage moves from v10 4 tasks/187 cells to v15_measured 7 tasks/1057 cells, while train coverage is 24 tasks/1187 cells. Calibration remains conditional: ARC shared scorer is a

calibration tradeoff: fixed-task NLL improves (4.0827 vs 8.2754) while ECE worsens (0.6995 vs 0.5589); OOD improves both NLL (2.6278 vs 4.3723) and ECE (0.4481 vs 0.4820).

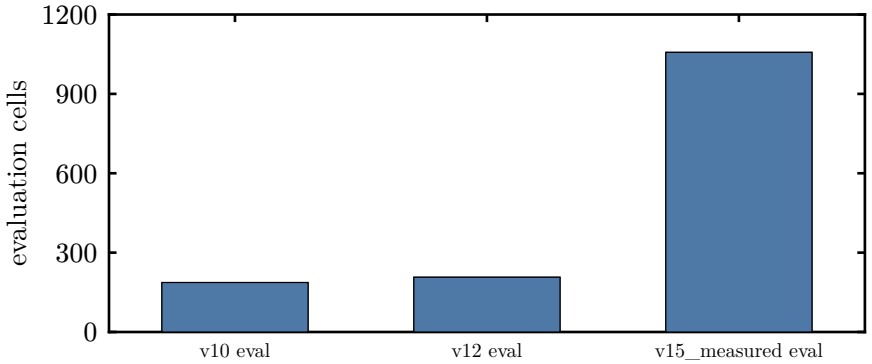


Figure 4: Public ARC evidence surface. This is measured `v15` coverage and failure taxonomy, not broad ARC competence.

Signal. To test physical control pressure, we use a simulator where bounded movement actions have recorded consequences. The model's WASD choices are scored against replayed simulation rows before live-control claims are admitted. This is the deterministic-replay pressure test. The live `neural_active` row is narrower: a running local Signal pilot applies checkpoint-ranked WASD proposals only through protocol checks and safety gates, and those gates may veto execution instead of letting the neural top choice own control. The trace records active source, checkpoint hash, latency, protocol compatibility, safety-gate rows, damage/death telemetry, and replay-ready frames: in offline Signal replay, the model ranks recorded WASD choices at top-1 79.76% vs baseline 67.86% (+11.90pp), utility 0.1432 vs 0.1359, regret 0.0073 vs 0.0146, damage 2.38% vs 2.38%, deaths 0.00% vs 0.00%; live shadow frames=74. the live Signal pilot did run checkpoint-ranked neural actions, but only behind guards: 74 recent neural-source rows from checkpoint f12b644ed9, p95 inference 0.089 ms, 74 replay-ready rows, protocol-compatible rows=74, safety-gate rows=74, damage events=2, deaths=0; no baseline comparison is claimed. The supported claim set still does not show that active neural control improves utility, regret, damage, or death versus the heuristic baseline.

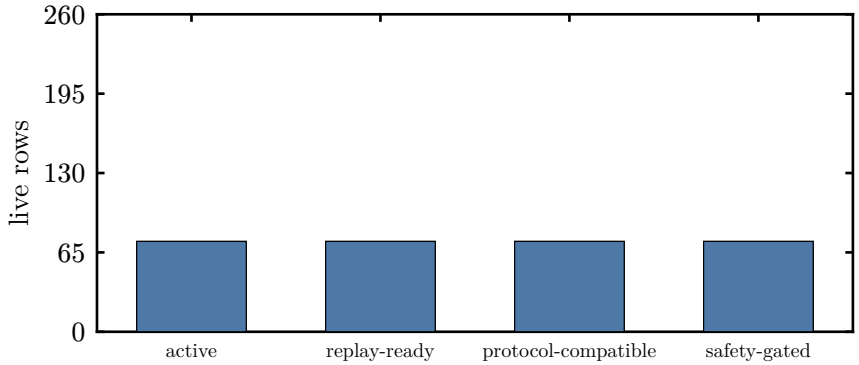


Figure 5: Signal guarded live neural-active execution. The row establishes live checkpoint-ranked execution and replay-ready telemetry, not comparative active-control superiority.

Architecture and language-adjacent probes. Some rows ask how much the design depends on model size or task-specific heads. Those are boundary checks, not architecture claims: C7 blocks this capacity claim until plateau audit is ready: the sweep's best fixed-ARC shared row is 32x16 at 39.20% and delayed_claim deltas are +0.10pp to +0.43pp, but only 20/72 plateau rows and 0/12 domain/variant verdicts are audit-ready. C7 blocks this adapter-head architecture claim until plateau audit is

ready: fixed ARC heads-flat delta +0.00pp; delayed_claim heads-flat +0.00pp; grid_path heads-flat +0.23pp; ARC OOD heads-single +5.80pp top-1 and -1.2020 NLL, with only 20/72 plateau rows and 0/12 domain/variant verdicts audit-ready. For haiku, there is no exact checker for meaning. The system can only check whether a line uses corpus-derived vocabulary, adjacency, syllable/shape constraints, and real-vs-corrupt discrimination. We call that weaker checker `corpus_proxy`; it makes haiku a boundary probe, not a semantic language-understanding result.

9. Claim Governance

Empirical reporting is wired directly to experiment outputs. Result scripts write canonical summaries, and the claim compiler turns registered source artifacts into Typst macros, support-surface rows, and limitation candidates. The compilation layer uses those macros rather than hand-maintained numbers. `make paper-support` refreshes support-owned audit artifacts such as trace determinism and dataset-engine evidence before checking the claim registry. It does not silently rerun the full promoted-result snapshot: canonical headline rows such as `results/latest/comparison.canonical.jsonl` are refreshed by `make results` and then checked by `make paper-support`. If an artifact is missing, stale relative to its declared freshness prerequisite, contradicted, or underpowered, the manuscript compilation layer will reject the registered claim as supported. The trace determinism row is currently supported: the trace audit checks whether replay meaning survives numeric/runtime changes: 2 traces and 2000 rows, 0 invalid rows, 0 rank-sensitive rows, 0 logged mismatches, same semantic hash: yes. The dataset-engine row is governed the same way: the dataset-first engine evidence surface has 8 promoted-smoke rows across 8 domains; 8/8 rows declare dataset, rule-search, replay, and evidence capabilities, 8/8 pass candidate-group, verifier-target, and checkpoint-identity contracts, 4 rows expose multi-target verifier labels, and the weakest smoke test top-1 is 60.00%. These rows are infrastructure evidence, not promoted domain-result claims. This does not mean every sentence is cryptographically enforced. It means the empirical claims registered with the claim compiler must stay attached to live artifacts.

Class	**Count**	**Body rule**
supported	14	May be interpreted, using generated summaries and artifact-backed scope.
contradicted	0	Must remain negative evidence, limitation, or claim-change work.
underpowered	4	May motivate roadmap items, not body-strength architecture claims.
missing/stale	0 / 0	Must block or remove body claims until artifacts are regenerated.

Table 5: Generated claim-gate summary. The detailed support-surface rows remain in `paper/generated/claims.typ`; the body uses counts and generated claim macros rather than forcing every long support row into the main text.

Editorial agents fit the same rule. They may improve wording, structure, transitions, and clarity through reviewable pull requests. They may not change claim scope, numeric results, generated summaries, citations, or experiment interpretation unless they open an explicit claim-change PR. The disclosure is operational: AI agents are assistants, while empirical claims come from repository artifacts checked before compilation.

10. Limitations

1. **Scope is deliberately narrow.** The supported object is a checked search system: rules decide validity, neural models rank what remains, and claims stay tied to artifacts. The formal scope is verifier-grounded residual ranking and executable claim governance. It does not support broad task solving, broad ARC competence, semantic language understanding, or competitive theorem proving.
2. **Multi-seed intervals are still uneven.** The clean next-prime filter headline now has a generated seed interval. Some boundary rows still need more explicit confidence intervals before they should be treated as stable quantitative laws.
3. **The fixed-budget proposer result is negative.** The generated static-proposer baseline claim is supported as a negative result; it is the reason rejected-candidate learning matters.
4. **Architecture claims are underpowered or contradicted.** Tiny-net plateau evidence remains underpowered, and adapter heads are diagnostic rather than promoted architecture.
5. **Signal is live but not comparative.** The live `neural_active` row establishes guarded checkpoint-ranked execution with replay-ready telemetry. It does not establish active neural improvement over the heuristic baseline.
6. **Corpus proxies are weaker than verifiers.** The haiku task can ground train-split adjacency, shape, and corruption discrimination. It cannot ground truth or semantics.
7. **Candidate-row scalability depends on bounded support.** Continuous action spaces and multi-agent settings need an explicit enumerator, sampler, or replayable proposal surface before the protocol applies. Without that surface, the engine can log and compare runs but cannot make verifier-grounded guarantees over the unconstrained action space.
8. **Feedback is artifact-bounded.** Rejected rows and certified hypotheses can train later rankers, but claim promotion depends on frozen traces, held-out checks, semantic hashes, and regenerated support reports. Online policy drift is evidence to audit, not a silent update to an existing claim.
9. **Engine ports are not domain promotions.** The dataset-first runner proves a shared row, replay, checkpoint, and evidence contract across smoke adapters. It does not replace the richer domain CLIs, rule searches, replay reports, or paper evidence needed to promote a domain result.
10. **Executable accountability is partial.** Claim compilation catches unsupported generated claims, stale evidence, and limitation candidates. It does not replace human judgment about interpretation, ethics, or whether a domain belongs in the supported claim set.

Claim	Status	Generated limitation candidate
C4.multitask_capacity_tradeoff	underpowered	C7 blocks this capacity claim until plateau audit is ready: the sweep's best fixed-ARC shared row is 32x16 at 39.20% and delayed_claim deltas are +0.10pp to +0.43pp, but only 20/72 plateau rows and 0/12 domain/variant verdicts are audit-ready.
C4.adapter_heads_shared_trunk	underpowered	C7 blocks this adapter-head architecture claim until plateau audit is ready: fixed ARC heads-flat delta +0.00pp; delayed_claim heads-flat +0.00pp; grid_path heads-flat +0.23pp; ARC OOD heads-single +5.80pp top-1 and -1.2020 NLL, with only 20/72 plateau rows and 0/12 domain/variant verdicts audit-ready.
C7.tiny_net_plateau_stop_rule	underpowered	tiny-net plateau audit covers 6 capacities, 3 restarts, and 8 checkpoints through 128 epochs across 12 domain/variant verdicts; 20/72 required rows meet the plateau threshold and 0/12 verdicts are audit-ready.
C5.arc_verified_self_play	underpowered	ARC verified self-play is underpowered: certified eval=-1, template=7, oracle=14, opened zero-coverage families=0, seeds=1, public replay=no, fixed-budget ablation=no, leakage gate=no. The claim remains L1 search efficiency, not new primitive capability.

Table 6: Generated limitation candidates from the claim compiler. The required claim support gate is currently passing. Optional contradicted and underpowered claims remain visible here instead of blocking PDF compilation.

11. Conclusion

The result is a disciplined loop. Enumerate candidates. Check which ones are allowed. Let a small model rank the allowed choices. Preserve the trace. Bind empirical claims to the generated evidence. Arithmetic sequence filtering supplies the clean certified-gate anchor, with the next-prime trace as the fully audited instance. Rejected-candidate learning and hypothesis-library promotion show that checked failures and checked successes can feed back into later proposal structure. The dataset-first engine shows how future experiments can enter the system as candidate rows plus verifier labels before becoming paper claims. ARC, Signal, tiny-net, adapter, and haiku rows mark the boundary rather than widening the thesis.

The broader lesson is pragmatic. When correctness, safety, or auditability matter, the model should not own the whole decision and prose should not outrun the evidence. The checker or replay system owns validity and consequence. The learned component owns ranking pressure. The repository trace owns what empirical reporting is allowed to say.

Bibliography

[1] A. Solar Lezama, “Program Synthesis By Sketching,” Doctoral dissertation, 2008. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-177.html

[2] R. Alur *et al.*, "Syntax-Guided Synthesis," in *Proceedings of the IEEE International Conference on Formal Methods in Computer-Aided Design (FMCAD)*, 2013, pp. 1–17.

[3] E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-Guided Abstraction Refinement," in *Computer Aided Verification (CAV)*, 2000, pp. 154–169. doi: 10.1007/10722167_15.

[4] J. P. Marques-Silva and K. A. Sakallah, "GRASP: A Search Algorithm for Propositional Satisfiability," *IEEE Transactions on Computers*, vol. 48, no. 5, pp. 506–521, 1999.

[5] R. J. Bayardo and R. Schrag, "Using CSP Look-Back Techniques to Solve Real-World SAT Instances," in *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI)*, 1997, pp. 203–208.

[6] K. Cobbe *et al.*, "Training Verifiers to Solve Math Word Problems," *arXiv preprint arXiv:2110.14168*, 2021.

[7] H. Lightman *et al.*, "Let's Verify Step by Step," *arXiv preprint arXiv:2305.20050*, 2023.

[8] J. Uesato *et al.*, "Solving math word problems with process- and outcome-based feedback," *arXiv preprint arXiv:2211.14275*, 2022.

[9] A. Hosseini, X. Yuan, N. Malkin, A. Courville, A. Sordoni, and R. Agarwal, "V-STaR: Training Verifiers for Self-Taught Reasoners," *arXiv preprint arXiv:2402.06457*, 2024.

[10] DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," *arXiv preprint arXiv:2501.12948*, 2025.

[11] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong, "Solving olympiad geometry without human demonstrations," *Nature*, vol. 625, pp. 476–482, 2024.

[12] Y. Chervonyi *et al.*, "Gold-medalist Performance in Solving Olympiad Geometry with AlphaGeometry2," *arXiv preprint arXiv:2502.03544*, 2025.

[13] S. Polu and I. Sutskever, "Generative Language Modeling for Automated Theorem Proving," *arXiv preprint arXiv:2009.03393*, 2020.

[14] M. Mitchell *et al.*, "Model Cards for Model Reporting," in *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, 2019, pp. 220–229. doi: 10.1145/3287560.3287596.

[15] T. Gebru *et al.*, "Datasheets for Datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021, doi: 10.1145/3458723.

[16] J. Pineau *et al.*, "Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

[17] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep Reinforcement Learning That Matters," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, 2018, pp. 3207–3214. doi: 10.1609/AAAI.V32I1.11694.